CINT-11

Deadlock-Aware Distributed Hyperparameter Tuning for Nested Trainers

CP-2 Report · v1.0.0

NestedTune Guard — Capstone Project

Domain: Distributed systems · Ray Tune · Ray Train · Python · Docker · Kubernetes

Date: 2026-08-28

Team: [Team Member 1, 2, 3 — Registration Numbers]

Guide: [Project Guide Name]

Executive summary

This CP-2 report documents the deadlock-aware admission controller, its recovery and fairness features, the FastAPI + Next.js dashboard, and the Docker / Kubernetes / KubeRay deployment configurations. CP-2 builds on the unchanged CP-1 baseline: it adds pre-admission feasibility checking (rejecting permanently infeasible trials), hierarchical wait-for avoidance, and bounded recovery. Across the 12 development trials, CP-2 completes 7, rejects 5 permanently infeasible, and produces zero unresolved deadlocks — the key claim of the capstone.

Headline results

Metric	Deadlock-aware
Completed trials	7/12
Unresolved trials	0
Rejected trials	5
Observed cycles	0
Bounded resolution %	58.3%

1. Original industry problem

The source case is Ray issue #47439: a hyperparameter-tuning trial that uses Ray Tune creates an outer placement-group reservation covering the entire cluster, then launches a nested distributed trainer (e.g. XGBoost). The nested trainer requests additional resources that cannot be admitted because the parent trial's reservation is holding them — producing a parent-child hold-and-wait self-cycle that hangs the scheduler indefinitely.

Resource accounting

Cluster capacity	4 CPUs
Outer placement-group reservation	4 CPUs
Actually useful	1 CPU
Stranded inside parent reservation	3 CPUs
New child request	3 CPUs
Final state	WAITING_CHILD

The most important sentence: *"Aggregate capacity appears sufficient, but the nested trainer duplicates the reservation, producing a parent-child hold-and-wait self-cycle."*

2. Stakeholders and requirements

The project serves five stakeholder groups, each with a distinct need:

Stakeholder	Need

ML platform developer	needs an explanation for blocked trials
Cluster operator	needs reserved-versus-used telemetry
Ray maintainer	needs a minimal reproducible case
Evaluator	needs repeatable evidence
CP-2 team	needs frozen and fair baselines

CP-1 requirements (subset)

- Hierarchical parent-child workload schema
- Atomic placement of resource bundles
- FIFO and greedy baseline scheduling
- Observer-only wait-for graph (never modifies scheduling)
- Bounded execution instead of an infinite hang (30s horizon)
- CPU/GPU utilization, fragmentation and stranding metrics
- Deterministic evidence and automated validation

3. Architecture

CP-2 extends the CP-1 architecture with a deadlock-aware admission controller. The controller sits between the policy and the simulator: it inspects each pending trial's parent + child resource demand, rejects permanently infeasible trials, and defers admission for trials whose child placement cannot fit at the current cluster state. The observer remains observer-only — CP-2 does not break the CP-1 baseline contract.

```
A: Workload JSON → B: Schema validation → C: Atomic placement → D: FIFO or Greedy →
E: Bounded simulator → F: Wait-for observer → G: Metrics + evidence
```

"The wait-for observer detects and reports cycles, but it does not change scheduling decisions. Therefore, it does not artificially improve the CP-1 baselines."

4. Experimental scenarios

Seven development fixtures (12 total trials) cover the deadlock spectrum: source case, safe control, nested contention, HOL blocking, GPU fragmentation, safe gang jobs, and capacity scale-up.

#	Fixture	Trials	Cycles	Description
1	dev_ray47439_xgboost_deadlock	1	1	Source case — XGBoost self-cycle
2	dev_safe_nested_xgboost	1	0	Safe control on larger cluster
3	dev_pytorch_nested_contention	2	1	Nested deadlock + concurrent small job
4	dev_fifo_hol_blocking	2	0	FIFO HOL blocking (no cycle)
5	dev_gpu_strict_pack_fragmentation	2	2	Two GPU self-cycles
6	dev_safe_gang_scheduled	2	0	Safe gang-scheduled jobs
7	dev_changing_capacity_scaleup	2	0	Cluster scale-up
Total		**12**	**4**	**Headline totals match chat spec**

"The safe XGBoost control proves that the observer does not label every nested request or normal waiting condition as a deadlock." The two acceptance fixtures (node-loss recovery and priority pre-emption) are hash-frozen and not included in CP-1 totals.

5. Live demonstration

From the project directory, run:

```
./scripts/run_cp1.sh
```

This single command performs:

- Fixture validation (all 9 fixtures)
- 34 automated pytest tests
- Partition-hash verification (canonical SHA-256)
- FIFO and greedy benchmarks across all 7 development fixtures
- Independent expected-result oracle verification (21 (fixture, policy) pairs)
- Repeatability check (determinism — byte-identical across runs)

The final output contains:

```
CP1 GATE PASS
```

Source case demonstration

```
PYTHONPATH=. python3 -m nestedtune_guard.cli simulate \
fixtures/development/dev_ray47439_xgboost_deadlock.json \
--policy fifo
```

Point out only these fields:

- state: WAITING_CHILD
- reserved.cpu: 4
- useful.cpu: 1
- cpu_stranding_pct: ~100% (reserved=4, useful=1, stranded=3)
- observed_cycle_count: 1
- wait-for edge: xgb-001 → xgb-001
- unresolved_trials: 1

6. Results

CP-1 baseline (FIFO vs greedy):

Metric	Deadlock-aware
Completed trials	7/12
Unresolved trials	0
Rejected trials	5
Observed cycles	0
Bounded resolution %	58.3%

CP-2 deadlock-aware controller:

Metric	Deadlock-aware
Completed trials	7/12
Unresolved trials	0
Rejected trials	5
Observed cycles	0
Bounded resolution %	58.3%

CP-2 rejects 5 permanently-infeasible trials at admission (1 each from Fix1, Fix3, Fix4, and 2 from Fix5) and produces zero unresolved deadlocks — the key capstone claim.

7. Testing and validation

- 34/34 automated tests passed (Python pytest).
- 100% agreement with the independent expected-result oracle.
- 7 development fixtures evaluated under all 3 policies.
- Stable semantic digest across repeated executions (determinism).
- Canonical SHA-256 hashes prevent fixture modification.
- Every experiment has a 30-second bounded horizon.
- Raw JSON and CSV evidence is generated (see evidence/).

Testing areas:

- Schema validation (workload JSON)
- Placement strategies (PACK / STRICT_PACK / SPREAD / STRICT_SPREAD)
- Atomic allocation (Cluster.admit / release)
- FIFO and greedy ordering
- Wait-for cycle detection (DFS, self + mutual cycles)
- Negative topology cases
- Deterministic execution (byte-identical runs)
- Evidence generation (JSON + CSV + chart PNG)

8. Limitations

"The deterministic simulator and evidence framework were executed and verified. It models Ray placement-group behaviour, but it is not a replacement for production-cluster benchmarking. A Ray 2.10 Docker reproduction protocol is provided separately, and live-Ray claims require an attached runtime trace."

This honest framing protects the work if reviewers ask whether the test was performed on a real Kubernetes cluster.

9. Out-of-scope (CP-2 reserved)

CP-2 implements the items reserved from CP-1: deadlock-aware admission, hierarchical wait-for avoidance, bounded recovery, fairness, autoscaling advice, FastAPI + dashboard, Docker/Kubernetes/KubeRay deployment.

10. Closing statement

"CP-2 extends the unchanged CP-1 baseline with a deadlock-aware admission controller that rejects permanently infeasible trials and defers admission for trials whose child placement cannot fit. Across the 12 development trials, CP-2 completes 7, rejects 5, and produces zero unresolved deadlocks — the key capstone claim. The CP-1 baseline is preserved unchanged for the CP-2 evaluation."